

Lab - 5: Creating Your First Bash Script

This week, we shall look at creating a script file, that can do multiple things, apart from simply using pre-existing commands. We will add functions that you can write yourself, check the return value and combine it with loops.

1. Setup

First, create a new directory called `lab-5` in your `home` directory (using `mkdir lab-5`) and change into it (using `cd lab-5/`). This will now be your working directory. It is always a good idea to create a clean working directory, when you are experimenting with new techniques.

Step 1: Create a New Script File

Open your terminal and create a new file called `my_script.sh`:

```
Unset
touch my_script.sh
```

Remember to use a *plain-text* editor (e.g., nano, pico, Notepad++, VSCode - *NOT* MS Word!!) to add code to this script. The “Unset” string that you see in the box is because my pdf maker does not recognize bash. Please ignore it.

Step 2: Add the Shebang

At the top of your script, add the shebang line to specify the interpreter:

```
Unset
#!/bin/bash
```

The `#` character is also called *hash* and the exclamation point (`!`) is also called *bang*. So, `#!` became shortened to *shebang* (the vocabulary of programmers and computer scientists is, well, strange). The path `/bin/bash` means that the shell executable (`bash`) resides in the directory called `/bin`. Check out the `/bin` directory to see what other kinds of executables are present. Incidentally, `bin` stands for `binaries`

Step 3: Make the Script Executable

Change the file permissions to make the script executable. To make your script executable, you must first change permissions on it. This can be done by running the command:

```
Unset  
chmod u+x ./my_script.sh
```

The command above, gave the user who owns the script (in this case, you) **execute** permissions.

Step 4: Write a Simple Script

Open the script in your text editor and add the following code:

```
Unset  
#!/bin/bash  
  
echo "Hello, World!"
```

Step 5: Run Your Script

Execute your script by running:

```
Unset  
./my_script.sh
```

You should see the output:

```
Unset  
Hello, World!
```

2. Using Functions in Bash Scripts

Functions allow you to group commands and reuse them throughout your script.

Step 1: Define a Function

Add a function to your script that greets the user:

```
Unset
#!/bin/bash

greet() {
  echo "Hello, $1!"
}

greet "Alice"
```

Step 2: Run the Script

Run your script again:

```
Unset
./my_script.sh
```

You should see:

```
Unset
Hello, Alice!
```

Step 3: Function with Return Value

You can also create functions that return values. For example, a function to add two numbers:

```
Unset
add() {
  local sum=$(( $1 + $2 ))
  echo $sum
}

result=$(add 5 10) # Note how the function was called. This is different from
how other programming languages conventionally do it.

echo "The sum is: $result"
```

3. Checking the Return Value of Commands

Every command in Bash returns an exit status, which can be checked using the special variable `$?`. An exit status of `0` indicates success, while any other value indicates an error.

Step 1: Check Command Success

Add the following code to your script to check if a command was successful:

```
Unset
#!/bin/bash

mkdir my_directory

if [ $? -eq 0 ]; then
echo "Directory created successfully."
else
echo "Failed to create directory."
fi
```

Step 2: Run the Script

Run your script again:

```
Unset
./my_script.sh
```

You should see:

```
Unset
Directory created successfully.
```

Step 3: Handling Errors

You can also handle errors more gracefully by using functions. Here's an example:

```
Unset
create_directory() {
  mkdir "$1"
  if [ $? -eq 0 ]; then
    echo "Directory '$1' created successfully."
  else
    echo "Failed to create directory '$1'."
  fi
}

create_directory "my_new_directory"
```

4. Putting It All Together

Here's a complete example script that combines everything we've learned:

```
Unset
#!/bin/bash

# Function to greet the user
greet() {
  echo "Hello, $1!"
}

# Function to add two numbers
add() {
  local sum=$(( $1 + $2 ))
  echo $sum
}

# Function to create a directory
create_directory() {
  mkdir "$1"
  if [ $? -eq 0 ]; then
    echo "Directory '$1' created successfully."
```

```
else
echo "Failed to create directory '$1'."
fi
}

# Main script execution
greet "Alice"
result=$(add 5 10)
echo "The sum is: $result"
create_directory "my_final_directory"
```

Step 1: Save and Run

Save the script and run it:

```
Unset
./my_script.sh
```

Expected Output

You should see output similar to:

```
Unset
Hello, Alice!
The sum is: 15
Directory 'my_final_directory' created successfully.
```

Exercise: Create a Bash Script with a Function to Calculate Factorials

Write a Bash script that defines a function to calculate the factorial of a given non-negative integer. The script should prompt the user to enter a non-negative integer, call the function to compute the factorial, and then display the result.

Requirements:

1. Define a function named `factorial` that takes one argument (a non-negative integer) and returns the factorial of that number.
2. The script should handle invalid input (negative numbers) by displaying an appropriate error message and prompting the user to enter a valid number again.

3. The script should continue to prompt the user until a valid non-negative integer is provided.
4. After calculating the factorial, the script should display the result in a user-friendly format.

Example Output:

```
Unset
Enter a non-negative integer: 5
The factorial of 5 is 120.

Enter a non-negative integer: -3
Invalid input.

Enter a non-negative integer: 4
The factorial of 4 is 24.
```

Hints:

- You can use the `read` command to get user input. Check the manpage for `read` if you need to
 - Use a conditional statement to check if the input is a non-negative integer.
 - Recall that a factorial for 'n' is defined as: $n * (n - 1) * (n - 2) \dots * 1$
 - If $n == 0$, then the factorial is 1. If $n == 1$, then the factorial is 1
-